— S I D H I L Y N X —

LUMETRIX

L E A R N I N G P L A T F O R M

PYTHON PROGRAMMING

C U R A T E D Q U E S T I O N B A N K

374 Questions | 10 Modules | 4 Difficulty Levels

Easy • Moderate • Hard • Capstone

Section A (Academic 40%) | Section B (Real-World 60%)

CONFIDENTIAL — FOR REVIEWER USE ONLY

Property of Lumetrix | Sidhilynx © 2026

Edition 1.0 | Academic Year 2025–26

Developed & Published by Lumetrix, a Sidhilynx Initiative

— S I D H I L Y N X —

LUMETRIX

L E A R N I N G P L A T F O R M

ABOUT THIS DOCUMENT

This question bank has been meticulously curated by the academic content team at Lumetrix, an educational initiative under Sidhilynx. It is designed to provide a structured, comprehensive, and industry-aligned resource for learners pursuing Python programming across academic and professional domains.

Spanning 10 modules and 374 carefully crafted questions, this bank bridges the gap between theoretical understanding and real-world application through its dual-section format.

ABOUT LUMETRIX

Lumetrix is a premier learning platform under the Sidhilynx ecosystem, committed to delivering high-quality, outcome-driven educational content. Our mission is to empower learners with skills that are directly applicable in the modern technology landscape.

DOCUMENT SCOPE & STRUCTURE

- 10 Modules covering the complete Python programming curriculum
- Section A: Academic questions aligned with university examination patterns
- Section B: Real-world scenario-based questions for industry readiness
- Four difficulty tiers: Easy, Moderate, Hard, and Capstone
- 10 Capstone projects integrating multiple modules for end-to-end practice

QUALITY ASSURANCE

This document undergoes a rigorous three-tier verification process before release. It is reviewed by a subject matter expert (Professor), validated by an industry practitioner (Real-World Engineer), and approved by the Chief Executive Officer of Sidhilynx.

INTELLECTUAL PROPERTY NOTICE

All content in this document is the exclusive intellectual property of Sidhilynx and Lumetrix. Unauthorized reproduction, distribution, or use of this material — in whole or in part — is strictly prohibited and may attract legal consequences under applicable IP laws.

Lumetrix | A Sidhilynx Initiative | www.sidhi.xyz | ceo@sidhi.xyz

— S I D H I L Y N X —

LUMETRIX

L E A R N I N G P L A T F O R M

TABLE OF CONTENTS

Module 1 — Computer Fundamentals & Problem Solving
Module 2 — Python Basics & Data Representation
Module 3 — Operators and Expressions
Module 4 — Conditional Control Statements
Module 5 — Repetitive Control Statements
Module 6 — Functions and Recursion
Module 7 — Strings and Tuples
Module 8 — Lists and List Comprehensions
Module 9 — Sets, Dictionaries & Functional Programming
Module 10 — Modules, Packages, Exceptions & File Handling
Capstones — Integrated Project Challenges (10 Projects)

— S I D H I L Y N X —

LUMETRIX

L E A R N I N G P L A T F O R M

DOCUMENT VERIFICATION & APPROVAL

This Question Bank has been independently reviewed, validated, and approved by the following authorities prior to official release on the Lumetrix platform.

ACADEMIC REVIEWER	INDUSTRY ENGINEER	CEO — SIDHILYNX
Name:	**Name:**	**Name:** **ARSHAD AZEEZ M**
Institution / Organisation:	**Institution / Organisation:**	**Organisation:** SIDHILYNX
Designation:	**Designation:**	**Designation:** CHIEF EXECUTIVE OFFICER
Signature: _____	**Signature:** _____	**Signature:** _____

APPROVED FOR RELEASE ON LUMETRIX PLATFORM

Verified for academic integrity, technical accuracy, and real-world relevance

— S I D H I L Y N X —

LUMETRIX

L E A R N I N G P L A T F O R M

Module 1: Computer Fundamentals & Problem Solving

Topics Covered:

- Computer architecture and components
- Problem-solving techniques
- Algorithms: definition and characteristics
- Pseudocode
- Flowcharts
- Introduction to Python interpreter (interactive and script mode)

Section A — Academic Questions

Easy (Section A)

Q1. **[Easy]** Write a Python program to find the largest of three numbers entered by a user.

Q2. **[Easy]** Write a program that checks whether a given number is even or odd and prints the result.

Q3. **[Easy]** Create a program that adds two numbers 15 and 27, then subtracts 10 from the result and displays the final output.

Q4. **[Easy]** Write a program that reads a number from the user and prints "Positive" if the number is greater than 0, otherwise prints "Non-positive".

Q5. **[Easy]** A student scores 78, 82, and 90 in three subjects. Write a program to calculate and display the average score.

Moderate (Section A)

Q6. **[Moderate]** Write a program that reads a student's marks in 4 subjects, calculates the total, and assigns a grade: Total ≥ 300: Grade A | Total ≥ 240: Grade B | Total ≥ 180: Grade C | Total < 180: Grade D.

Q7. **[Moderate]** Create a program to find the factorial of a number N using a loop. Test with N=5 and N=7.

Q8. **[Moderate]** Write a program that reads marks of 30 students and counts how many students scored above 75.

Q9. **[Moderate]** Create a program to calculate the area of a circle. The program should read the radius from the user and use $\pi = 3.14159$.

Q10. **[Moderate]** Write a program to check if a year entered by the user is a leap year. (Hint: divisible by 4 but not by 100, OR divisible by 400)

Q11. **[Moderate]** Write a program that reads a student's attendance percentage and decides: ≥ 75%:

"Eligible for exam" | < 75%: "Not eligible - Low attendance"

Hard (Section A)

Q12. **[Hard]** Create a program to find the GCD (Greatest Common Divisor) of two numbers using the Euclidean algorithm. Test with inputs 48 and 18.

Section B — Real-World Questions

Easy (Section B)

Q13. **[Easy]** A coffee vending machine charges ₹20 per cup. Write a program that takes the number of cups and calculates the total bill.

Q14. **[Easy]** Create an ATM withdrawal program that checks if balance is sufficient. If balance ≥ withdrawal amount, allow withdrawal and show new balance, otherwise deny.

Q15. **[Easy]** A parking lot charges ₹30 for the first hour and ₹20 for each additional hour. Write a program to calculate parking fees for H hours entered by user.

Q16. **[Easy]** Create a simple calculator program that takes two numbers and an operator (+, -, *, /) and performs the operation.

Q17. **[Easy]** A shop offers 10% discount if purchase amount > ₹1000. Write a program to calculate final payable amount.

Q18. **[Easy]** Write a program that converts currency from INR to USD using exchange rate 1 USD = ₹83.

Q19. **[Easy]** A fitness tracker counts steps. Write a program that reads daily steps for 7 days and calculates the weekly total and daily average.

Moderate (Section B)

Q20. **[Moderate]** Create an e-commerce shipping calculator: Order < ₹500: ₹50 shipping | Order ₹500-₹1000: ₹30 shipping | Order > ₹1000: Free shipping.

Q21. **[Moderate]** Write a temperature monitoring program that reads 24 hourly temperature readings and counts how many times temperature exceeded 35°C. Also find the maximum temperature recorded.

Q22. **[Moderate]** Create a movie ticket booking system: Weekday ₹150/ticket | Weekend ₹200/ticket | Senior citizens (age ≥ 60): 20% discount.

Q23. **[Moderate]** Write a program that validates whether a mobile number is correct: Must be exactly 10 digits | First digit must be 6, 7, 8, or 9.

Q24. **[Moderate]** Write a program to calculate electricity bill: 0-100 units ₹5/unit | 101-300 units ₹7/unit | 301+ units ₹10/unit. Also add ₹50 as fixed meter charge.

Hard (Section B)

Q25. **[Hard]** Write a program to find the maximum profit from stock prices. Given a list of 7 daily prices, find the best day to buy and best day to sell to maximize profit. Example: [100, 95, 110, 90, 105, 115, 85] → Buy on day 4 (₹90), Sell on day 6 (₹115), Profit = ₹25.

Q26. **[Hard]** Create a password strength checker that evaluates a password in ONE pass: Check for uppercase letter, lowercase letter, digit, special character | Length ≥ 8 | Print strength: Weak (0-2 criteria), Medium (3 criteria), Strong (4+ criteria).

Module 2: Python Basics & Data Representation

Topics Covered:

- Introduction to Python programming language
- Python program structure
- Identifiers and keywords
- Variables and data representation
- Built-in data types
- Input and output statements

Section A — Academic Questions

Easy (Section A)

Q27. **[Easy]** Write a program that demonstrates the use of all basic data types in Python: int, float, str, bool. Create one variable of each type and print them with their types using type().

Q28. **[Easy]** Create a program that takes a student's name and roll number as input, then displays them in the format: "Student Name: [name], Roll No: [number]"

Q29. **[Easy]** Write a program that declares variables for length and breadth of a rectangle, calculates area and perimeter, and displays the results with appropriate labels.

Q30. **[Easy]** Create a program that takes three subject marks as input (float values) and prints them rounded to 2 decimal places.

Q31. **[Easy]** Write a program that demonstrates the difference between print() statements with and without the sep and end parameters.

Q32. **[Easy]** Write a program that takes a sentence as input and prints: the sentence in uppercase | the sentence in lowercase | the length of the sentence.

Q33. **[Easy]** Create a program that demonstrates variable reassignment. Start with x = 10, then x = 20, then x = "Hello". Print value and type after each assignment.

Moderate (Section A)

Q34. **[Moderate]** Write a program that takes radius as input, calculates both area and circumference of a circle, and displays results formatted to 3 decimal places.

Q35. **[Moderate]** Create a program that demonstrates type conversion: Take a float input (45.67), convert to int | Take an int input (100), convert to float and string.

Q36. **[Moderate]** Write a program that takes marks in 5 subjects as input, calculates total and percentage, then prints a formatted marksheet with Subject-wise marks, Total, and Percentage.

Q37. **[Moderate]** Create a program that reads a 4-digit number and separates it into individual digits using integer division and modulo operations.

Q38. **[Moderate]** Write a program that takes base and height of a triangle as float inputs, calculates area, and handles the output formatting to show exactly 2 decimal places.

Q39. **[Moderate]** Write a program that reads temperature in Celsius as float, converts to Fahrenheit and Kelvin, and displays all three temperatures in a formatted table structure.

Q40. **[Moderate]** Create a program that takes a 6-digit student ID number as input and extracts: First 2 digits: Year of admission | Next 2 digits: Department code | Last 2 digits: Roll number.

Hard (Section A)

Q41. **[Hard]** Write a program that validates user input for age: Keep asking for age until a valid integer is entered | Display "Invalid input" for non-numeric input | Check if age is between 1 and 120 | Use try-except for error handling.

Q42. **[Hard]** Create a program that demonstrates the limitations of float data type: Add 0.1 + 0.2 and print result | Check if result equals 0.3 | Show how to properly compare floats using round() or checking difference.

Q43. **[Hard]** Write a program that takes a large number (e.g., 1234567890) and formats it with commas as thousand separators, in scientific notation, and as a string with custom formatting using f-strings.

Section B — Real-World Questions

Easy (Section B)

Q44. **[Easy]** Create a simple invoice generator that takes product name, quantity, and price per unit as input, then displays Product, Quantity, Price per unit, and Total.

Q45. **[Easy]** Write a program for a digital weighing scale that takes weight in kilograms (float) and converts it to Grams and Pounds (1 kg = 2.20462 lbs).

Q46. **[Easy]** Create a program that takes customer name and phone number as input, then displays a confirmation message.

Q47. **[Easy]** Write a currency denomination counter: Take an amount in rupees (integer), then calculate how many ₹500, ₹100, ₹50, ₹20, ₹10 notes are needed.

Q48. **[Easy]** Create a simple time converter that takes time in minutes and converts it to hours and minutes format. Example: Input 135 → Output "2 hours 15 minutes"

Q49. **[Easy]** Write a program for a fuel efficiency calculator: Input distance traveled (km) and fuel consumed (liters) | Output: Mileage in km/l formatted to 2 decimal places.

Q50. **[Easy]** Create a simple tip calculator: Input bill amount | Calculate 10%, 15%, and 20% tip options | Display all three options.

Q51. **[Easy]** Write a program for a juice shop that takes number of glasses sold and displays: Small (₹30), Medium (₹50), Large (₹70) totals and Total Revenue.

Moderate (Section B)

Q52. **[Moderate]** Create a BMI calculator with formatted output: Input name, weight (kg), height (m) | Calculate BMI | Display a personalized report with Name, Weight, Height, and BMI.

Q53. **[Moderate]** Write a program for an online shopping cart: Input product name, original price, discount percentage | Calculate discount amount and final price | Display all details formatted with ₹ symbol.

Q54. **[Moderate]** Create a gym membership calculator: Input member name, membership type (monthly/quarterly/yearly) | Rates: Monthly ₹1000, Quarterly ₹2700 (10% off), Yearly ₹9600 (20% off).

Q55. **[Moderate]** Write a program that validates and formats phone numbers: Input 10-digit number | Check if exactly 10 digits | Format as +91-XXXXX-XXXXX.

Q56. **[Moderate]** Create an age calculator: Input birth year | Calculate current age (use 2024 as current year) | Calculate age in months and days (approximate).

Q57. **[Moderate]** Write a comprehensive salary slip generator: Input employee name, employee ID, basic salary | Calculate HRA (40%), DA (20%), PF (12%) | Display formatted salary slip.

Q58. **[Moderate]** Create a scientific calculator for quadratic equation: Input a, b, c in $ax^2 + bx + c$ | Calculate discriminant | Show nature of roots | Handle a = 0 case.

Q59. **[Moderate]** Write a program for a hotel booking system: Input guest name, room number, nights, room rate, is_member (bool) | Members get 15% discount | Add 18% GST | Display complete booking confirmation.

Module 3: Operators and Expressions

Topics Covered:

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Expressions and operator precedence
- Program execution and debugging basics

Section A — Academic Questions

Easy (Section A)

Q60. **[Easy]** Write a program that takes two numbers and demonstrates all arithmetic operators (+, -, *, /, //, %, **). Display each operation with its result.

Q61. **[Easy]** Create a program that takes a number and prints its square, cube, and square root using appropriate operators.

Q62. **[Easy]** Write a program that takes two integers and compares them using all relational operators (>, <, >=, <=, ==, !=). Print True/False for each comparison.

Q63. **[Easy]** Create a program that demonstrates the difference between / (division) and // (floor division) with the same pair of numbers. Test with 17 and 5.

Q64. **[Easy]** Write a program that takes a number and checks if it's divisible by both 3 and 5 using the modulo operator.

Q65. **[Easy]** Create a program using compound assignment operators (+=, -=, *=, /=). Start with x = 10, perform all operations.

Q66. **[Easy]** Write a program that takes a number and checks if it's positive, negative, or zero using relational operators.

Moderate (Section A)

Q67. **[Moderate]** Write a program that takes coefficients a, b, c of a quadratic equation and calculates the discriminant (b^2 - 4ac). Use this to determine if roots are real, equal, or imaginary.

Q68. **[Moderate]** Create a program that takes a 4-digit number and reverses it using only arithmetic operators (no string conversion).

Q69. **[Moderate]** Write a program that takes exam marks and attendance percentage, then checks eligibility: (marks >= 40) and (attendance >= 75). Print detailed reason if not eligible.

Q70. **[Moderate]** Create a program demonstrating operator chaining in comparisons: Check if 10 < x < 100 for a given x.

Q71. **[Moderate]** Write a program that takes a student's score (0-100) and uses relational operators to check: Excellent (>=90), Good (>=75), Average (>=50), Fail (<50).

Q72. **[Moderate]** Create a program that evaluates De Morgan's Laws: Verify not(A and B) == (not A) or (not B) and not(A or B) == (not A) and (not B). Take boolean inputs for A and B.

Q73. **[Moderate]** Write a program that takes total seconds and converts to hours:minutes:seconds format using only //, % operators.

Hard (Section A)

Q74. **[Hard]** Create a program that solves a quadratic equation $ax^2 + bx + c = 0$ completely: Calculate discriminant | Find both roots | Handle all cases (real distinct, real equal, complex).

Q75. **[Hard]** Write a program that evaluates operator precedence in complex expressions: 2 + 3 * 4 ** 2 / 8 - 5 % 3 | Show step-by-step evaluation.

Q76. **[Hard]** Write a program that checks multiple conditions for loan approval: Age between 21-60 | Income > 25000 OR co-applicant income > 15000 | Credit score >= 700 | Existing loans < 3 | Combine all using logical operators.

Section B — Real-World Questions

Easy (Section B)

Q77. **[Easy]** Write a program for a retail shop that calculates: Subtotal (price * quantity) | Tax (subtotal * 0.18) | Total (subtotal + tax).

Q78. **[Easy]** Create a program that checks if a customer is eligible for a discount: Purchase amount >= 1000. Use relational operators.

Q79. **[Easy]** Write a program for a pizza delivery system that adds ₹40 delivery charge if order amount < 300, otherwise free delivery.

Q80. **[Easy]** Create a program that calculates electricity bill: Units consumed * ₹7 + ₹50 (fixed charge).

Q81. **[Easy]** Write a program that checks if a user's age is valid for driving license (age >= 18). Use relational operators.

Q82. **[Easy]** Create a program for a taxi service: Base fare ₹50 + ₹12 * distance_km. Calculate total fare.

Q83. **[Easy]** Write a program that calculates discount amount: Original price * (discount_percentage / 100).

Moderate (Section B)

Q84. **[Moderate]** Write a program for an e-commerce site that calculates final price: Apply discount if amount > 1000 | Add shipping if amount < 500 | Add GST (18%).

Q85. **[Moderate]** Create a program for credit card payment: Minimum payment = 5% of outstanding OR ₹500 (whichever is higher). Calculate both, use relational operators to select maximum.

Q86. **[Moderate]** Write a program for insurance premium calculation: Base premium ₹10000 | Add 20% if age < 25 OR age > 60 | Add 15% if smoker | Subtract 10% if regular_exercise.

Q87. **[Moderate]** Create a program for a telecom billing system: Free up to 100 minutes | ₹1 per minute for 101-300 minutes | ₹0.50 per minute beyond 300.

Q88. **[Moderate]** Write a program for overtime pay calculation: Regular hours (<=40): rate * hours | Overtime hours (>40): rate * 1.5 * extra_hours | Weekend hours: rate * 2 * weekend_hours.

Q89. **[Moderate]** Create a program for traffic fine calculation: Base fine ₹500 | Add ₹100 for every 10 km/h above limit | Double fine if school zone | Triple fine if repeat offender.

Hard (Section B)

Q90. **[Hard]** Create a comprehensive loan approval system: Credit score >= 750 OR (>= 650 AND income > 50000) | DTI ratio < 40% | Employment >= 2 years | No defaults in last 2 years.

Q91. **[Hard]** Write a program for dynamic pricing algorithm: Base price | Demand factor (high/medium/low: +30%/0%/-20%) | Inventory factor (low stock: +15%, high stock: -10%) | Member: -10%, New customer: +5%.

Q92. **[Hard]** Create a program for a stock trading profit/loss calculator: Buy price, sell price, quantity, brokerage (0.5%) | Short term capital gains tax (15%) if held < 1 year | Long term (10%) on profit > ₹100000 if held >= 1 year.

Module 4: Conditional Control Statements

Topics Covered:

- Simple if statement
- if-else statement
- if-elif-else ladder
- Nested conditional statements
- Branching statements: break, continue, pass

Section A — Academic Questions

Easy (Section A)

Q93. [Easy] Write a program that takes a number and checks if it's positive using a simple if statement. Print "Positive number" if true.

Q94. [Easy] Create a program that takes a number and prints whether it's even or odd using if-else.

Q95. [Easy] Write a program that takes a student's marks and prints "Pass" if marks >= 40, otherwise prints "Fail" using if-else.

Q96. [Easy] Create a program that takes two numbers and prints which one is greater using if-elif-else.

Q97. [Easy] Write a program that takes a character and checks if it's a vowel (a, e, i, o, u) using simple if statements.

Q98. [Easy] Create a program that takes age as input and prints "Eligible to vote" if age >= 18, otherwise "Not eligible".

Q99. [Easy] Write a program that takes a number and checks if it's divisible by 5 using if-else.

Q100. [Easy] Create a program that takes three numbers and finds the largest using if-elif-else.

Q101. [Easy] Write a program that takes a percentage and assigns grade: A (>=90), B (>=80), C (>=70), D (>=60), F (<60).

Moderate (Section A)

Q102. [Moderate] Write a program that takes three angles and checks if they can form a valid triangle (sum = 180 and all > 0) using nested if statements.

Q103. [Moderate] Create a program that classifies a triangle as Equilateral, Isosceles, or Scalene based on three side lengths.

Q104. [Moderate] Write a program that implements a simple calculator (+, -, *, /) using if-elif-else. Handle division by zero.

Q105. [Moderate] Create a program that checks if a number is divisible by 2, 3, or 5, and prints all applicable divisibilities using multiple if statements (not elif).

Q106. [Moderate] Write a program that takes a number from 1-12 and prints the month name and number of days in that month (assume non-leap year).

Q107. [Moderate] Create a program that takes height and weight, calculates BMI, and categorizes as Underweight (<18.5), Normal (18.5-24.9), Overweight (25-29.9), Obese (>=30).

Q108. [Moderate] Write a program that takes exam marks and attendance percentage, then applies: If marks >= 40 AND attendance >= 75: Pass | If marks >= 40 BUT attendance < 75: Detained | If marks < 40: Fail.

Q109. [Moderate] Create a program that validates a date (day, month, year) considering leap years for February using nested if-else.

Hard (Section A)

Q110. [Hard] Create a program that solves a quadratic equation ($ax^2 + bx + c = 0$) with complete handling: Check if a = 0 | Calculate discriminant | Handle two real roots, one real root, complex roots.

Q111. [Hard] Write a program that validates a complex password with multiple criteria: Length >= 8 | At least one uppercase | At least one lowercase | At least one digit | At least one special character.

Q112. [Hard] Create a program for admission eligibility with nested conditions: Age: 17-25 for general, 17-28 for reserved | Marks: >=60% for general, >=55% for reserved | Entrance score: >=50 for general, >=45 for reserved.

Q113. [Hard] Write a program that implements a game of "Guess the Number": 5 attempts | After each guess: "Too high", "Too low", or "Correct" | Use break when guessed correctly.

Section B — Real-World Questions

Easy (Section B)

Q114. [Easy] Write a program for a shop discount system: If purchase >= 1000, apply 10% discount. Otherwise, no discount. Print final amount using if-else.

Q115. [Easy] Create a program for ATM withdrawal: Check if balance >= withdrawal_amount. If yes, process; if no, deny.

Q116. [Easy] Write a program for a parking system: Cars pay ₹50, Bikes pay ₹20. Take vehicle type and print parking fee.

Q117. [Easy] Create a program for a coffee shop: Small-₹30, Medium-₹50, Large-₹70. Take size input and print price.

Q118. [Easy] Write a program for age-based movie ticket pricing: Child (<12): ₹100, Adult: ₹200, Senior (>=60): ₹150.

Q119. [Easy] Create a program that checks if a customer qualifies for free shipping (order >= ₹500) using if-else.

Q120. [Easy] Write a program for a restaurant that adds 10% service charge if bill > ₹500 using if-else.

Q121. [Easy] Create a program for a taxi service: Day fare (6AM-10PM): ₹15/km, Night fare: ₹20/km. Take time and distance, calculate fare.

Q122. [Easy] Write a program for a library fine: If days late > 0, charge ₹5/day. Otherwise, no fine.

Q123. [Easy] Create a program that checks if a user is eligible for credit card (age >= 21 AND income >= 20000).

Moderate (Section B)

Q124. [Moderate] Write a program for a tiered discount system: < ₹500: No discount | ₹500-₹1000: 5% | ₹1001-₹2000: 10% | >₹2000: 15%.

Q125. **[Moderate]** Create a program for movie ticket booking: Weekday ₹150 | Weekend ₹200 | Senior citizen (age >= 60): 20% discount on any day. Use nested if-else.

Q126. **[Moderate]** Write a program for a progressive tax calculator: 0-250000: 0% | 250001-500000: 5% | 500001-1000000: 20% | >1000000: 30%.

Q127. **[Moderate]** Create a program for insurance premium: Base ₹10000 | Age < 25 OR age > 60: add 20% | Smoker: add 15% | Regular exercise: subtract 10%.

Q128. **[Moderate]** Write a program for a restaurant billing system: Calculate food bill | If bill > ₹1000: 10% discount | Add 5% GST | Add 10% service charge.

Q129. **[Moderate]** Create a program for scholarship distribution: CGPA >= 9.0: ₹50000 | 8.0-9.0: ₹30000 | 7.0-8.0: ₹15000 | Family income must be < ₹500000.

Q130. **[Moderate]** Write a program for traffic violation fine system: Speeding ₹1000 + ₹100 per 10 km/h over limit | No helmet/seatbelt ₹500 | Signal jump ₹1000 | Drunk driving ₹10000.

Q131. **[Moderate]** Create a program for a hotel booking system: Standard ₹2000, Deluxe ₹3500, Suite ₹6000 | Weekend: add 30% | Peak season: add 40% | Member: 15% discount.

Hard (Section B)

Q132. **[Hard]** Write a comprehensive student result processing system: Take marks in 5 subjects | Check if any subject < 35 (fail) | If passed all, calculate percentage | Award: Distinction >= 75% with no subject < 60 | First class >= 60% with no subject < 50 | Second class >= 50% | Pass class >= 40%.

Q133. **[Hard]** Create a comprehensive loan eligibility system with nested conditions: Check multiple loan types (Home/Personal/Car) | Each with different eligibility criteria (age, income, credit score) | Calculate maximum loan amount | Determine interest rate based on credit score.

Q134. **[Hard]** Write a program for an airline dynamic pricing system: Base price by class (Economy/Business/First) | Weekday vs Weekend pricing | Advance booking discount (>30 days: 20%, >15 days: 10%) | Festival season surge (40%) | Group booking (5+ passengers): 15% discount.

Q135. **[Hard]** Create a comprehensive student result processing system using deeply nested conditionals handling all cases: marks, attendance, grace marks, improvement exam eligibility.

Module 5: Repetitive Control Statements

Topics Covered:

- for loop
- while loop
- Nested loops
- Loop control techniques
- Pattern generation programs

Section A — Academic Questions

Easy (Section A)

Q136. **[Easy]** Write a program using a for loop to print numbers from 1 to 10.

Q137. **[Easy]** Create a program using a while loop to print numbers from 10 to 1 (reverse order).

Q138. **[Easy]** Write a program to calculate the sum of first N natural numbers using a for loop.

Q139. **[Easy]** Create a program to print the multiplication table of a given number using a for loop.

Q140. **[Easy]** Write a program using a while loop to find the factorial of a number.

Q141. **[Easy]** Create a program to print all even numbers from 1 to 50 using a for loop.

Q142. **[Easy]** Write a program to count the number of digits in a given number using a while loop.

Q143. **[Easy]** Create a program to print the first N Fibonacci numbers using a for loop.

Q144. **[Easy]** Write a program using a while loop to reverse a given number.

Q145. **[Easy]** Create a program to print all numbers from 1 to 20, but skip multiples of 3 using continue statement.

Moderate (Section A)

Q146. **[Moderate]** Write a program using nested for loops to print the pattern: 1 / 1 2 / 1 2 3 / 1 2 3 4 / 1 2 3 4 5.

Q147. **[Moderate]** Create a program using a while loop to check if a number is prime.

Q148. **[Moderate]** Write a program using nested for loops to print a multiplication table from 1 to 10 (10x10 grid).

Q149. **[Moderate]** Create a program using a for loop to find all prime numbers between 1 and 100.

Q150. **[Moderate]** Write a program using nested for loops to print the pyramid pattern: * / *** / ***** / ******* / *********.

Q151. **[Moderate]** Create a program using a for loop to check if a number is an Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$).

Q152. **[Moderate]** Write a program using nested for loops to print Floyd's triangle: 1 / 2 3 / 4 5 6 / 7 8 9 10.

Q153. **[Moderate]** Create a program using nested loops to print a diamond pattern.

Q154. **[Moderate]** Write a program using a for loop to calculate the sum of the series: 1 + 1/2 + 1/3 + ... +

1/N.

Hard (Section A)

Q155. **[Hard]** Write a program using a while loop to find the nth Fibonacci number using only two variables (space-optimized).

Q156. **[Hard]** Create a program using nested loops to print all perfect numbers between 1 and 1000.

Q157. **[Hard]** Write a program using a for loop to implement the Sieve of Eratosthenes algorithm to find all primes up to N.

Q158. **[Hard]** Create a program using nested for loops to print the complex pattern: 1 / 212 / 32123 / 4321234 / 543212345.

Q159. **[Hard]** Write a program using a for loop with multiple control variables to find all Pythagorean triplets ($a^2 + b^2 = c^2$) where a, b, c < 50.

Section B — Real-World Questions

Easy (Section B)

Q160. **[Easy]** Write a program using a for loop to calculate the total bill for a restaurant where N items are ordered. Take price of each item as input.

Q161. **[Easy]** Create a program using a while loop for an ATM that keeps allowing withdrawals until balance becomes zero or user chooses to exit.

Q162. **[Easy]** Write a program using a for loop to calculate the total marks of N students.

Q163. **[Easy]** Create a program using a while loop for a login system that gives 3 attempts to enter the correct password.

Q164. **[Easy]** Write a program using a for loop to calculate total sales for a week (7 days).

Q165. **[Easy]** Create a program using a while loop to keep accepting product prices until user enters -1, then display the total.

Q166. **[Easy]** Write a program using a for loop to calculate total electricity units consumed over 12 months.

Q167. **[Easy]** Create a program using a while loop to simulate rolling a dice until a 6 appears, count the number of rolls.

Q168. **[Easy]** Write a program using a while loop to keep accepting ratings (1-5) until user enters 0, then calculate average rating.

Q169. **[Easy]** Create a program using a while loop for a game where player keeps guessing until they get the correct number.

Q170. **[Easy]** Write a program using a for loop to print all seat numbers in a theater (A1 to A20, B1 to B20 for 5 rows).

Moderate (Section B)

Q171. **[Moderate]** Write a program using nested for loops to generate a seating chart for a classroom with R rows and C columns.

Q172. **[Moderate]** Create a program using a while loop to calculate compound interest for N years, showing year-by-year breakdown.

Q173. **[Moderate]** Write a program using a for loop with break to find the first month where sales exceed a target amount.

Q174. **[Moderate]** Create a program using a while loop to simulate a bank account with deposits and withdrawals, continuing until balance < minimum balance.

Q175. **[Moderate]** Write a program using a for loop with continue to process orders, skipping cancelled orders and calculating total revenue.

Q176. **[Moderate]** Create a program using nested loops to generate a report card for N students with M subjects each.

Q177. **[Moderate]** Write a program using a for loop to calculate daily, weekly, and monthly sales targets and track progress.

Q178. **[Moderate]** Create a program using a while loop to simulate a stock trading session, buying/selling until target profit is reached.

Q179. **[Moderate]** Write a program using nested loops to print a ticket booking layout for a cinema (rows and seats).

Hard (Section B)

Q180. **[Hard]** Create a program using nested loops to simulate a complete e-commerce order processing system: Multiple customers placing multiple orders | Calculate order total, apply discounts, add shipping | Track inventory and update stock | Generate daily sales report.

Q181. **[Hard]** Write a program using loops to implement a complete student grading system: N students, M subjects each | Calculate individual totals, percentages, grades | Find class topper, subject toppers | Calculate class average for each subject | Generate rank list.

Q182. **[Hard]** Create a program using nested for loops to generate a complete monthly calendar for any given month and year: Calculate starting day of the week | Print dates in proper grid format | Handle leap years | Mark weekends differently.

Q183. **[Hard]** Write a program using loops to implement a banking transaction system: Multiple account holders | Different transaction types (deposit, withdrawal, transfer) | Maintain transaction history | Calculate interest on savings | Generate monthly statement.

Module 6: Functions and Recursion

Topics Covered:

- Defining and calling functions
- Function arguments and return values
- Scope and namespaces
- Recursive functions
- Lambda functions

Section A — Academic Questions

Easy (Section A)

Q184. [Easy] Write a function greet() that takes a name as parameter and prints "Hello, [name]!". Call it with different names.

Q185. [Easy] Create a function add_two_numbers(a, b) that returns the sum of two numbers. Test it with multiple values.

Q186. [Easy] Write a function is_even(num) that returns True if a number is even, False otherwise.

Q187. [Easy] Create a function square(n) that returns the square of a number. Call it in a loop for numbers 1 to 10.

Q188. [Easy] Write a function max_of_two(a, b) that returns the maximum of two numbers without using the built-in max().

Q189. [Easy] Create a function print_table(n) that prints the multiplication table of n.

Q190. [Easy] Write a function celsius_to_fahrenheit(c) that converts and returns temperature in Fahrenheit.

Q191. [Easy] Create a function greet_default(name="Guest") that uses a default parameter value.

Q192. [Easy] Write a function swap(a, b) that returns two values in swapped order. Use tuple unpacking when calling.

Q193. [Easy] Create a function count_vowels(text) that counts and returns the number of vowels in a string.

Moderate (Section A)

Q194. [Moderate] Write a recursive function factorial_recursive(n) that calculates factorial. Compare its output with the iterative version.

Q195. [Moderate] Create a function fibonacci(n) that returns the nth Fibonacci number using recursion.

Q196. [Moderate] Write a function gcd(a, b) that finds the GCD of two numbers using recursion (Euclidean algorithm).

Q197. [Moderate] Create a function using **kwargs to print all keyword arguments: print_details(**kwargs).

Q198. [Moderate] Write a function with both *args and **kwargs: flexible_function(*args, **kwargs).

Q199. [Moderate] Create a function demonstrating variable scope: define a global variable, modify it

using global keyword inside a function.

Q200. **[Moderate]** Write a recursive function binary_search(arr, target, low, high) that implements binary search.

Q201. **[Moderate]** Create a higher-order function apply_operation(func, num) that takes a function and a number, applies the function.

Hard (Section A)

Q202. **[Hard]** Write a recursive function tower_of_hanoi(n, source, destination, auxiliary) that solves the Tower of Hanoi puzzle and prints moves.

Q203. **[Hard]** Create a function implementing the Sieve of Eratosthenes algorithm to find all primes up to n efficiently.

Q204. **[Hard]** Write a recursive function quicksort(arr) that implements the quicksort algorithm.

Q205. **[Hard]** Create a function demonstrating closure: a function that returns another function with access to outer function's variables.

Q206. **[Hard]** Write a function implementing memoization for Fibonacci: fibonacci_memo(n, memo={}) to optimize recursive calls.

Section B — Real-World Questions

Easy (Section B)

Q207. **[Easy]** Write a function calculate_bill(items, prices) that takes lists of items and prices, returns total bill.

Q208. **[Easy]** Create a function apply_discount(price, discount_percent) that returns the discounted price.

Q209. **[Easy]** Write a function calculate_tax(amount, tax_rate=0.18) with default tax rate that returns amount after tax.

Q210. **[Easy]** Create a function bmi_calculator(weight, height) that calculates and returns BMI.

Q211. **[Easy]** Write a function simple_interest(principal, rate, time) that calculates and returns simple interest.

Q212. **[Easy]** Create a function convert_currency(amount, exchange_rate) that converts currency.

Q213. **[Easy]** Write a function calculate_age(birth_year, current_year=2024) that returns age.

Q214. **[Easy]** Create a function tip_calculator(bill, tip_percent=15) that calculates tip amount.

Q215. **[Easy]** Write a function parking_fee(hours) that calculates parking charges (₹50 first hour, ₹30 additional).

Moderate (Section B)

Q216. **[Moderate]** Write a function calculate_salary(basic, *allowances, **deductions) that uses both *args and **kwargs.

Q217. **[Moderate]** Create a recursive function calculate_compound_interest_recursive(principal, rate, time) for compound interest.

Q218. **[Moderate]** Write a function progressive_tax(income) that calculates tax based on multiple income slabs.

Q219. **[Moderate]** Create a function with nested helper functions: process_transaction(amount, transaction_type) with internal validation.

Q220. **[Moderate]** Write a function bulk_pricing(quantity, unit_price, discounts) where discounts is a dict of quantity tiers.

Q221. **[Moderate]** Create a function demonstrating closure: create_discount_calculator(default_discount) that returns a specialized calculator.

Q222. **[Moderate]** Write a function payroll_processor(employees) where employees is a list of dicts, calculates total payroll.

Q223. **[Moderate]** Create a function dynamic_pricing(base_price, demand_factor, time_factor, **modifiers) for dynamic pricing.

Q224. **[Moderate]** Write a recursive function calculate_depreciation_recursive(value, rate, years) for asset depreciation.

Hard (Section B)

Q225. **[Hard]** Write a comprehensive function e_commerce_checkout(cart, user_profile, coupon_codes, payment_method) that calculates item totals, applies discounts, validates coupons, adds shipping, calculates tax, and returns detailed breakdown. Use nested functions for each step.

Q226. **[Hard]** Create a recursive function organizational_salary_calculator(employee_tree) that calculates total salary for a hierarchy where employees can have subordinates.

Q227. **[Hard]** Write a function loan_amortization_schedule(principal, rate, tenure) that generates complete amortization schedule using recursion and returns list of payment details.

Q228. **[Hard]** Create a function dynamic_inventory_pricing(inventory, demand_data, competitor_prices) that uses closures to create pricing strategies and returns optimized pricing dict.

Module 7: Strings and Tuples

Topics Covered:

- String literals and methods
- String slicing and formatting
- String iteration and operations
- Tuples and tuple operations
- Nested tuples

Section A — Academic Questions

Easy (Section A)

Q229. **[Easy]** Write a program that takes a string and prints its length using len().

Q230. **[Easy]** Create a program that converts a string to uppercase and lowercase using string methods.

Q231. **[Easy]** Write a program that takes a string and checks if it starts with a specific character using startswith().

Q232. **[Easy]** Create a program that concatenates two strings using the + operator and prints the result.

Q233. **[Easy]** Write a program that reverses a string using slicing [::-1].

Q234. **[Easy]** Create a program that checks if a substring exists in a string using the in operator.

Q235. **[Easy]** Write a program that replaces all occurrences of 'a' with 'o' in a string using replace().

Q236. **[Easy]** Create a program that splits a sentence into words using split().

Q237. **[Easy]** Write a program that removes leading and trailing whitespaces from a string using strip().

Moderate (Section A)

Q238. **[Moderate]** Write a program that counts vowels and consonants in a string separately.

Q239. **[Moderate]** Create a program that checks if a string is a palindrome (case-insensitive).

Q240. **[Moderate]** Write a program that removes all vowels from a string.

Q241. **[Moderate]** Create a program that finds the most frequent character in a string.

Q242. **[Moderate]** Write a program that checks if two strings are anagrams of each other.

Q243. **[Moderate]** Create a program that performs string slicing with negative indices to extract the last 3 characters.

Q244. **[Moderate]** Write a program that implements Caesar cipher encryption (shift each letter by N positions).

Q245. **[Moderate]** Create a program that creates a nested tuple structure: ((1, 2), (3, 4), (5, 6)) and accesses inner elements.

Hard (Section A)

Q246. **[Hard]** Write a program that finds the longest palindromic substring in a given string.

Q247. **[Hard]** Create a program that implements the KMP pattern matching algorithm to find all

occurrences of a pattern in text.

Q248. **[Hard]** Write a program that implements a string similarity checker using Levenshtein distance algorithm.

Section B — Real-World Questions

Easy (Section B)

Q249. **[Easy]** Write a program that validates a phone number format: must be 10 digits, starts with 6/7/8/9.

Q250. **[Easy]** Create a program that extracts the domain name from an email address using string methods.

Q251. **[Easy]** Write a program that formats a product price with currency symbol: "Price: ₹1,234.56"

Q252. **[Easy]** Create a program that generates a username from a full name (first letter of first name + last name, lowercase).

Q253. **[Easy]** Write a program that validates a password: minimum 8 characters, contains uppercase and lowercase.

Q254. **[Easy]** Create a program that extracts hashtags from a social media post.

Q255. **[Easy]** Write a program that formats a date string from "YYYY-MM-DD" to "DD/MM/YYYY".

Q256. **[Easy]** Create a program that generates an acronym from a phrase: "Central Processing Unit" → "CPU".

Moderate (Section B)

Q257. **[Moderate]** Write a program that validates and formats an Aadhaar number: 12 digits, formatted as XXXX-XXXX-XXXX.

Q258. **[Moderate]** Create a program that parses a CSV line (comma-separated values) into a tuple, handling quoted fields.

Q259. **[Moderate]** Write a program that implements a simple search engine: finds documents containing all search terms (case-insensitive).

Q260. **[Moderate]** Create a program that generates a receipt format with items, prices, and total (aligned properly).

Q261. **[Moderate]** Write a program that validates credit card numbers using the Luhn algorithm.

Q262. **[Moderate]** Create a program that parses log entries: extracts timestamp, level (INFO/ERROR), and message.

Q263. **[Moderate]** Write a program that implements name matching with fuzzy logic (handles typos, abbreviations).

Hard (Section B)

Q264. **[Hard]** Write a comprehensive program that validates and formats multiple types of Indian documents: PAN Card | Aadhaar | Driving License | Passport. Uses proper error messages and formatting.

Q265. **[Hard]** Create a program that implements a spell checker: Loads dictionary (tuple of words) | Finds misspelled words | Suggests corrections using edit distance.

Q266. **[Hard]** Write a program that implements a templating system for email/SMS: Handles nested placeholders | Conditional sections | Loops for lists | Escaping special characters.

Module 8: Lists and List Comprehensions

Topics Covered:

- List creation and manipulation
- List operations
- Lists as stacks and queues
- Nested lists
- List comprehensions
- Matrix operations using lists

Section A — Academic Questions

Easy (Section A)

Q267. [Easy] Write a program that creates a list of 5 integers and prints it.

Q268. [Easy] Create a program that appends an element to a list using append() and inserts at a specific position using insert().

Q269. [Easy] Write a program that removes an element by value using remove() and by index using pop().

Q270. [Easy] Create a program that sorts a list in ascending and descending order.

Q271. [Easy] Write a program that implements a stack using list (push using append, pop using pop).

Q272. [Easy] Create a program that implements a queue using list (enqueue using append, dequeue using pop(0)).

Q273. [Easy] Write a list comprehension to create a list of even numbers from 1 to 20.

Q274. [Easy] Create a list comprehension to convert a list of strings to uppercase.

Q275. [Easy] Write a program that creates a 2D list (3x3) and prints it.

Q276. [Easy] Create a program that removes duplicates from a list while maintaining order.

Moderate (Section A)

Q277. [Moderate] Write a program that implements bubble sort algorithm on a list.

Q278. [Moderate] Create a program that implements binary search on a sorted list.

Q279. [Moderate] Write a program that creates a frequency dictionary from a list (element: count).

Q280. [Moderate] Create a program that rotates a 2D matrix 90 degrees clockwise.

Q281. [Moderate] Write a program that transposes a matrix (swap rows and columns).

Q282. [Moderate] Create a program that multiplies two matrices.

Q283. [Moderate] Write a program using nested list comprehension to create a multiplication table (N x N).

Q284. [Moderate] Create a program that finds all pairs in a list that sum to a target value.

Q285. [Moderate] Write a nested list comprehension to generate Pascal's triangle up to N rows.

Hard (Section A)

Q286. **[Hard]** Write a program that implements quicksort algorithm recursively on a list.

Q287. **[Hard]** Create a program that implements merge sort algorithm recursively.

Q288. **[Hard]** Write a program that solves the N-Queens problem and returns all solutions as nested lists.

Q289. **[Hard]** Create a program that finds the maximum sum subarray (Kadane's algorithm).

Q290. **[Hard]** Write a program that implements the longest increasing subsequence algorithm.

Section B — Real-World Questions

Easy (Section B)

Q291. **[Easy]** Write a program that stores shopping cart items in a list and calculates total price.

Q292. **[Easy]** Create a program that maintains a to-do list with add, remove, and display functions.

Q293. **[Easy]** Write a program that stores student names in a list and sorts them alphabetically.

Q294. **[Easy]** Create a program that stores daily temperatures for a week in a list and finds the average.

Q295. **[Easy]** Write a program that stores exam scores and calculates pass/fail count.

Q296. **[Easy]** Create a program that stores phone contacts as a list of lists: [name, number].

Q297. **[Easy]** Write a program using list comprehension to filter products above a certain price.

Q298. **[Easy]** Create a program that stores monthly expenses in a list and calculates yearly total.

Q299. **[Easy]** Write a program that manages a simple inventory: [item_name, quantity, price].

Moderate (Section B)

Q300. **[Moderate]** Write a program that implements a shopping cart with quantity updates and discount calculations using lists.

Q301. **[Moderate]** Create a program that manages student grades: nested lists for multiple subjects, calculates GPA.

Q302. **[Moderate]** Write a program that implements inventory management with low-stock alerts.

Q303. **[Moderate]** Create a program that analyzes sales data: finds best-selling products, monthly trends.

Q304. **[Moderate]** Write a program that manages flight bookings: seats as 2D list, booking/cancellation.

Q305. **[Moderate]** Create a program that analyzes stock prices: moving averages, trend detection.

Q306. **[Moderate]** Write a program that manages library book lending with due dates and fines.

Q307. **[Moderate]** Create a program that tracks fitness goals with progress monitoring.

Hard (Section B)

Q308. **[Hard]** Write a comprehensive inventory management system that: Tracks stock levels across multiple warehouses (nested lists) | Implements automatic reordering | Handles stock transfers between locations | Generates low-stock alerts | Produces inventory valuation reports.

Q309. **[Hard]** Create a complete student management system that: Stores student data in nested lists | Manages courses, enrollments, grades | Calculates GPAs with credit hours | Generates transcripts | Implements scholarship eligibility | Produces class rankings.

Q310. **[Hard]** Write a comprehensive e-commerce order fulfillment system that: Manages orders, inventory, shipping | Implements order batching | Optimizes warehouse picking routes | Handles partial shipments | Calculates shipping costs with zones.

Module 9: Sets, Dictionaries & Functional Programming

Topics Covered:

- Sets: creation and operations
- Dictionaries: creation and operations
- Dictionary iteration
- Dictionary comprehensions
- Functional programming concepts
- map(), filter(), reduce() functions

Section A — Academic Questions

Easy (Section A)

Q311. **[Easy]** Write a program that creates a set with 5 elements and demonstrates add(), remove(), and discard().

Q312. **[Easy]** Create a program that finds the union, intersection, and difference between two sets.

Q313. **[Easy]** Write a program that converts a list to a set to remove duplicates.

Q314. **[Easy]** Create an empty dictionary, add key-value pairs, update, and delete using pop() and del.

Q315. **[Easy]** Write a program using map() to square all numbers in a list.

Q316. **[Easy]** Create a program using filter() to get even numbers from a list.

Moderate (Section A)

Q317. **[Moderate]** Write a program that counts the frequency of each element in a list using a dictionary.

Q318. **[Moderate]** Create a program that groups elements by a property using dictionaries (e.g., group numbers by odd/even).

Q319. **[Moderate]** Write a program that inverts a dictionary (swap keys and values).

Q320. **[Moderate]** Create a dictionary comprehension to create a dictionary of squares: {1: 1, 2: 4, 3: 9, ...}.

Q321. **[Moderate]** Write a program using reduce() to find the sum and product of a list (import from functools).

Q322. **[Moderate]** Create a program using filter() and map() together in a pipeline.

Hard (Section A)

Q323. **[Hard]** Write a program that implements a complete LRU (Least Recently Used) cache using dictionaries and sets.

Q324. **[Hard]** Create a program that implements a trie data structure using nested dictionaries.

Section B — Real-World Questions

Easy (Section B)

Q325. **[Easy]** Write a program that stores unique customer IDs in a set and product inventory in a dictionary.

Q326. **[Easy]** Create a program that uses a set to track visited pages on a website.

Q327. **[Easy]** Write a program that stores employee IDs and salaries in a dictionary and finds the highest paid.

Q328. **[Easy]** Create a program using sets to find common skills among job candidates.

Q329. **[Easy]** Write a program using map() to apply discount to all prices in a list.

Q330. **[Easy]** Create a program using filter() to get customers with purchases above a threshold.

Moderate (Section B)

Q331. **[Moderate]** Write a program that implements a shopping cart using dictionaries: {product_id: {name, price, quantity}}.

Q332. **[Moderate]** Create a program that analyzes customer purchase patterns using sets and dictionaries.

Q333. **[Moderate]** Write a program that implements user preferences management with nested dictionaries.

Q334. **[Moderate]** Create a program using map() and reduce() for batch data processing of sales records.

Q335. **[Moderate]** Write a program that implements a voting system with candidate tracking using dictionaries.

Hard (Section B)

Q336. **[Hard]** Write a comprehensive e-commerce platform using sets and dictionaries that: Manages product catalog with categories | Implements shopping cart with pricing rules | Tracks inventory across warehouses | Implements recommendation engine | Uses functional programming for order processing pipeline.

Q337. **[Hard]** Create a complete social media analytics system that: Tracks user connections and followers (sets) | Analyzes engagement metrics (dictionaries) | Implements trending hashtags detection | Uses map(), filter(), reduce() for data aggregation.

Module 10: Modules, Packages, Exceptions & File Handling

Topics Covered:

- Importing standard and user-defined modules
- Creating and using packages
- Types of errors: syntax and logical errors
- Exception types and handling
- Multiple exception handling
- File handling: file paths, file modes
- Reading and writing files

Section A — Academic Questions

Easy (Section A)

Q338. **[Easy]** Write a program that imports the math module and uses sqrt(), ceil(), floor() functions.

Q339. **[Easy]** Create a program that imports random module and generates a random number between 1 and 100, and selects a random element from a list.

Q340. **[Easy]** Write a program that handles division by zero using try-except.

Q341. **[Easy]** Create a program that uses try-except-else-finally block demonstrating all four clauses.

Q342. **[Easy]** Write a program that opens a file in write mode, writes content, then reads it back.

Moderate (Section A)

Q343. **[Moderate]** Write a program that creates a package structure with __init__.py and imports modules from it.

Q344. **[Moderate]** Create a program that implements a custom exception class inheriting from Exception.

Q345. **[Moderate]** Write a program that reads a CSV file with headers using csv.DictReader and writes using csv.DictWriter.

Q346. **[Moderate]** Create a program that uses json module to read and write JSON data, handling malformed JSON errors.

Q347. **[Moderate]** Write a program that merges multiple text files into one and splits a large file into smaller files.

Hard (Section A)

Q348. **[Hard]** Write a comprehensive package with multiple modules demonstrating: Package hierarchy with subpackages | __init__.py with package-level imports | Relative imports | __all__ for controlling exports.

Q349. **[Hard]** Create a program that implements a robust file parser with: Multiple format support (JSON, CSV, XML) | Comprehensive error handling | Validation and sanitization | Progress reporting.

Section B — Real-World Questions

Easy (Section B)

Q350. [Easy] Write a program that reads customer data from a CSV file and prints it.

Q351. [Easy] Create a program that saves user preferences to a JSON file and loads them back.

Q352. [Easy] Write a program that logs error messages to a file with timestamps.

Q353. [Easy] Create a program that reads product inventory from a file and handles FileNotFoundError.

Q354. [Easy] Write a program that exports data to CSV format and backs up files to another directory.

Q355. [Easy] Create a program that saves shopping cart data to a file and loads it on program start.

Moderate (Section B)

Q356. [Moderate] Write a program that implements a customer database using CSV files with CRUD operations.

Q357. [Moderate] Create a program that processes uploaded files with validation (size, type, content).

Q358. [Moderate] Write a program that implements a file-based cache for API responses.

Q359. [Moderate] Create a program that validates and processes financial transaction files.

Q360. [Moderate] Write a program that implements a document versioning system.

Q361. [Moderate] Create a program that generates invoices from template files with data merge.

Hard (Section B)

Q362. [Hard] Write a comprehensive e-commerce order processing system that: Imports orders from multiple file formats (CSV, JSON, XML) | Validates order data with custom exceptions | Processes payments with error handling | Generates invoices and packing slips | Maintains audit logs.

Q363. [Hard] Create a complete HR and payroll system that: Imports employee data from various sources | Processes attendance and leaves | Calculates salaries with complex rules | Generates payslips and tax documents | Handles exceptions and corrections.

Q364. [Hard] Write a comprehensive enterprise integration platform that: Implements modular architecture with packages | Handles multiple file formats | Custom exception hierarchy | Robust error handling with retry and fallback | Comprehensive logging with rotation | Data validation framework.

CAPSTONE CHALLENGES

The following capstone projects are designed for extended sessions (60-120 minutes). They integrate multiple modules and require architectural thinking, not just coding.

Capstone 1: Complete Banking System

Modules: Modules 2, 5, 6, 8, 9

Build a complete banking system that manages multiple account types (savings, current, fixed deposit). Implement: customer registration with validation, deposit/withdrawal with daily limits, fund transfers between accounts, interest calculation (simple and compound), loan eligibility checker, EMI calculator, fraud detection (flag suspicious transactions), generate monthly statement. Use functions, loops, lists for account storage, dictionaries for transactions, file handling for persistence, and custom exceptions for error management.

Capstone 2: Student Management & Examination System

Modules: Modules 2, 4, 5, 6, 7, 8

Build a complete student management system: Student registration with ID generation, course enrollment with prerequisite checking, attendance tracking (flag < 75%), marks entry for multiple subjects with validation (0-100), automated grade calculation (A+/A/B+/B/C/F), grace marks policy (failed by <=5 marks AND overall > 45%), result processing (Distinction/First Class/Second Class/Pass/Fail), generate formatted report cards, rank list generation, scholarship eligibility checker. Use nested lists for data storage, functions for each module, file handling for data persistence.

Capstone 3: E-Commerce Platform

Modules: Modules 4, 5, 6, 7, 8, 9

Build a mini e-commerce platform: Product catalog management (add/remove/update products using dictionaries), shopping cart (add items, update quantity, remove), dynamic pricing engine (base price + demand factor + inventory factor + member discount), coupon code validation, checkout process (apply discounts → calculate tax → add shipping → final bill), inventory management with auto-reorder alerts, order history tracking, product recommendations based on purchase history using sets, generate itemized invoice. Handle all edge cases with proper exception handling.

Capstone 4: Hospital Management System

Modules: Modules 4, 5, 6, 8, 9, 10

Build a hospital management system: Patient registration with unique ID, appointment booking (check doctor availability, avoid conflicts using sets), queue management for different departments (emergency priority), doctor schedule management, prescription tracking with drug interaction checking (using tuple database), bed allocation system (2D list for floor/room), billing system (room charges + doctor fees + medicines + tests, insurance coverage), generate patient bill and discharge summary. Store data in files, use custom exceptions for medical errors.

Capstone 5: Inventory & Supply Chain System

Modules: Modules 5, 6, 8, 9, 10

Build an inventory and supply chain management system: Multi-warehouse stock tracking (nested dictionaries with location: {product: quantity}), supplier management, automatic purchase order generation when stock < reorder level, goods receipt processing, stock transfer between warehouses, FIFO inventory valuation, ABC analysis (A: top 20% value, B: next 30%, C: remaining 50%), demand forecasting using moving averages, generate inventory valuation report and stock movement log. Import/export data from CSV files, handle all file and data exceptions.

Capstone 6: Real Estate Management Platform

Modules: Modules 4, 6, 7, 8, 9, 10

Build a real estate platform: Property listing management (residential/commercial/land) with nested dictionaries, property search with multiple filters (location, type, price range, area), EMI calculator for home loans with complete amortization schedule, rental management (rent tracking, lease agreements, maintenance requests), agent commission calculator (tiered: 2% first 50L, 1.5% next 50L, 1% above 1Cr + GST 18% + agent/broker split), property valuation using multiple models, generate property report and transaction history. Store all data in JSON files.

Capstone 7: HR & Payroll Processing System

Modules: Modules 4, 5, 6, 8, 9, 10

Build a complete HR and payroll system: Employee database management (personal info, department, designation, salary grade), attendance tracking with leave management (casual/sick/earned leave with carry-forward rules), salary calculation engine (basic + HRA 40% + DA 20% + TA + special allowances - PF 12% - ESI - professional tax - TDS based on income slab), performance review system with rating-based increment calculation, bonus calculation (festival bonus + performance bonus), generate payslip and Form 16 equivalent. Multi-file data management with proper exception handling.

Capstone 8: Tournament Management System

Modules: Modules 4, 5, 6, 7, 8

Build a complete sports tournament manager: Support multiple formats (round-robin, knockout, group stage + knockout), team registration with player roster management, fixture generation avoiding scheduling conflicts, match result entry and validation, points table calculation (wins: 3pts, draw: 1pt, loss: 0pts) with tiebreaker rules (goal difference, head-to-head), progression to knockout rounds based on standing, player statistics tracking (goals, assists, yellow/red cards), generate match schedule and standings table. Handle all edge cases (walkovers, abandoned matches, etc.).

Capstone 9: Financial Portfolio Analyzer

Modules: Modules 3, 5, 6, 8, 9, 10

Build a financial portfolio analyzer: Multi-asset portfolio management (stocks, bonds, gold, FD, mutual funds), historical price tracking with moving averages (10-day, 50-day, 200-day), portfolio performance metrics (absolute return, CAGR, Sharpe ratio concept), risk assessment (volatility, beta), tax calculation on gains (STCG 15% if < 1 year for equity, LTCG 10% on profit > ₹1L if >= 1 year), dividend tracking, rebalancing recommendations when allocation deviates from target, generate comprehensive portfolio report. Load/save portfolio data from JSON files.

Capstone 10: Smart Restaurant System

Modules: Modules 4, 5, 6, 8, 9, 10

Build a complete smart restaurant management system: Menu management with categories and dynamic pricing (happy hours -30%, peak hours +20%, weekend +15%, festival +25%), table booking system with time slot management (avoid conflicts using sets), order management per table (add items, modify, cancel), kitchen queue optimization (priority by order time and table priority), bill generation (food + GST 5% + service charge 10% + applicable discounts - loyalty points), split bill option, daily sales analytics (revenue, popular items, peak hours, table turnover), inventory depletion tracking per order. Full file-based data persistence.

Total Questions: 374

Empowering Learners. Shaping Futures.

Platform: Lumetrix — lumetrix.sidhi.xyz

Parent Organisation: Sidhilynx — www.sidhi.xyz

Contact: ceo@sidhi.xyz

Edition: 1.0 | Academic Year 2025–26

Document Classification: Academic Internal — Confidential

Total Questions: 374 | 10 Modules | 10 Capstones

© 2026 Sidhilynx. All Rights Reserved.

Unauthorized reproduction or distribution of this document is strictly prohibited.